医学搜索词与疾病相关性判断

信息学院2023年秋大数据科学与工程

成员：苏莹、张志祥、宋银州、田炳河

时间：2023年12月3日

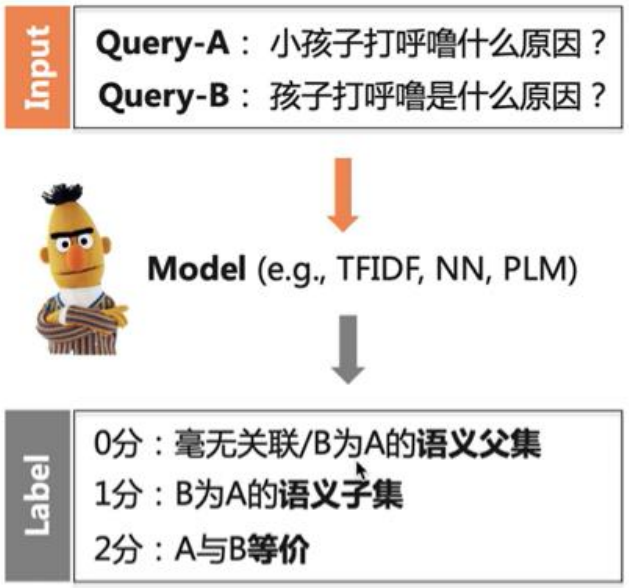
任务提纲

- 一、背景
- 二、模型
- 三、实现
- 四、其他思考

一、背景

- 通过分析用户病因搜索词，通过模型来分析与疾病的关联性

在搜索的业务中，判定两个Query（即查询词）之间的相关性是一项重要的任务，常用于长尾query的搜索质量优化场景。



2分：Query-A和Query-B语义等价

Query-A	Query-B
小孩子打呼噜是什么原因引起的	小孩子打呼噜什么原因
双眼皮怎么遗传	双眼皮遗传规律
黄体	女性黄体

1分：B为A的语义子集，B指代范围小于A

Query-A	Query-B
双眼皮遗传规律	内双眼皮遗传
尿酸高手脚酸痛	尿酸高 脚疼
海绵状血管瘤	多发性海绵状血管瘤

0分：B为A的语义父集，B指代范围大于A；或A与B语义毫无相关

Query-A	Query-B
死了的大闸蟹能吃吗	死了的螃蟹能吃么
拔智齿后悔死了	拔了智齿
光子嫩肤后注意事项	光子嫩肤的注意事项

•难点：对于语义理解和关系判断

二、模型

- before BERT
- BERT模型

三、实现

（一）文本预处理

中文分词及预处理

```
def _tokenize(self, text):
    tokens = list(jieba.cut(text))

    token_ids = []
    for token in tokens:
        if token in self.vocab_mapping:
            token_id = self.vocab_mapping[token]
            token_ids.append(token_id)
        else:
            # OOV
            if len(token) > 1:
                for t in list(token):
                    if t in self.vocab_mapping:
                        token_ids.append(self.vocab_mapping[t])
                    else:
                        token_ids.append(np.random.choice(len(self.vocab_mapping), 1)[0])
            else:
                token_ids.append(np.random.choice(len(self.vocab_mapping), 1)[0])

    token_ids, attention_mask = self._pad_truncate(token_ids)

    return token_ids, attention_mask
```

Step 1: 中文分词
（使用 **Jieba** 分词工具）

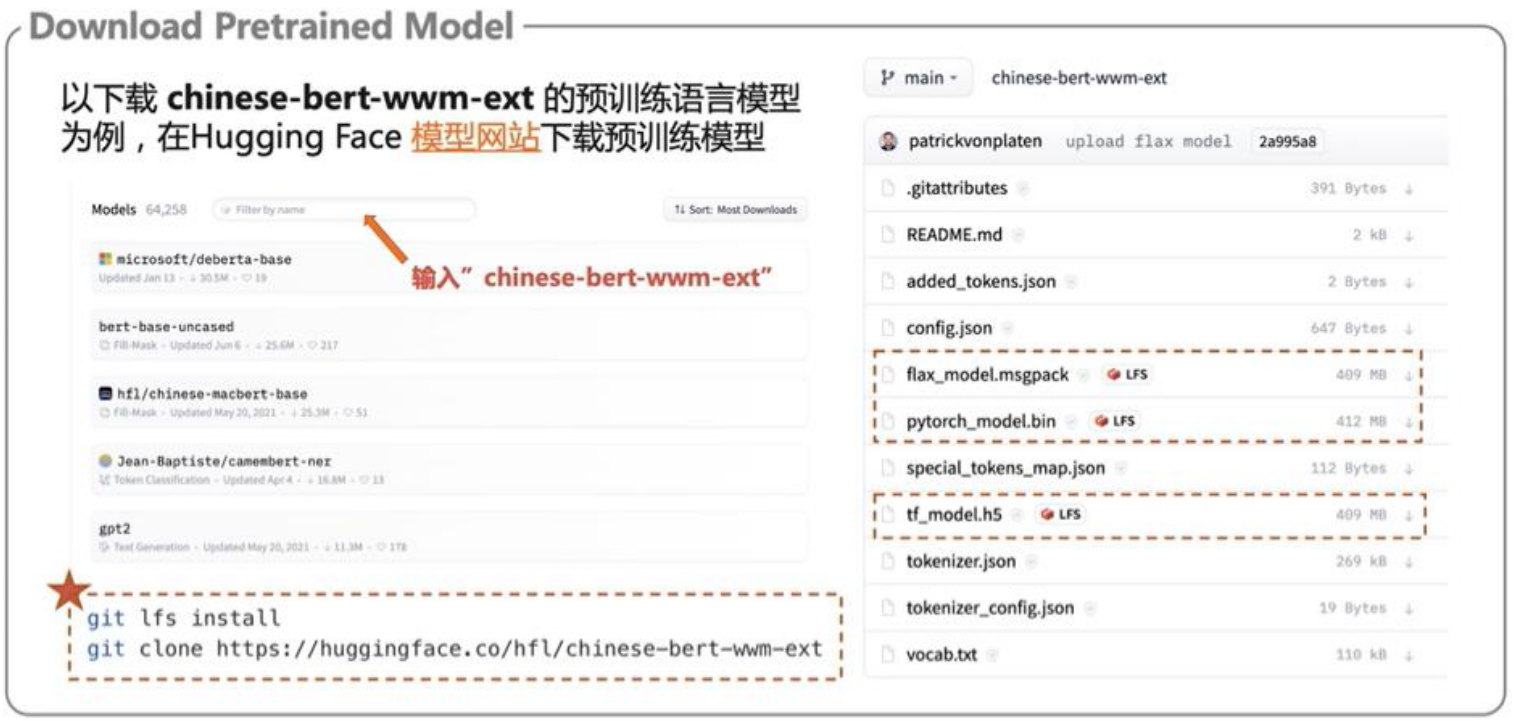
Step 2.1: 单词编码
（将单词映射为词向量模型的词表中对应单词的ID）

Step 2.2: 处理 Out of Vocabulary (OOV) 情况
（对于单字单词不在词表的情况，随机采样词表中一个单词代替；
对于多字单词，拆分成多个单字单词，并重复Step 2.1和Step 2.2）

Step 3: 文本序列填充或截断
（将处理好的单词序列填充至预设的最大长度，超出最大长度进行截断；小于最大长度进行以0填充）

三、实现

（一）文本预处理


Download Pretrained Model
以下载 chinese-bert-wwm-ext 的预训练语言模型为例，在Hugging Face 模型网站下载预训练模型
Models 64,258
Filter by name
Sort: Most Downloads
microsoft/deberta-base
输入" chinese-bert-wwm-ext"
bert-base-uncased
hfl/chinese-macbert-base
Jean-Baptiste/camembert-ner
gpt2
git lfs install
git clone https://huggingface.co/hfl/chinese-bert-wwm-ext
main
chinese-bert-wwm-ext
patrickvonplaten upload flax model 2a995a8
.gitattributes 391 Bytes
README.md 2 kB
added_tokens.json 2 Bytes
config.json 647 Bytes
flax_model.msgpack LFS 409 MB
pytorch_model.bin LFS 412 MB
special_tokens_map.json 112 Bytes
tf_model.h5 LFS 409 MB
tokenizer.json 269 kB
tokenizer_config.json 19 Bytes
vocab.txt 110 kB

三、实现

（一）文本预处理

Tokenizer

Step 1: 载入预训练模型的tokenizer

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained('<MODEL_DIR>')
```

Step 2: 分词与编码

```
encoded_input = tokenizer(
    text=['婴儿多大可以吃蛋清？'],
    text_pair=['几个月的宝宝可以吃蛋清'],
    padding='max_length',
    max_length=32,
    truncation=True,
    return_tensors='pt',
)
print(encoded_input)

"""
{'input_ids': tensor([[ 101, 2048, 1036, 1914, 1920, 1377,  809, 1391, 6028, 3926, 8043,  102,
  1126, 702, 3299, 4638, 2140, 2140, 1377,  809, 1391, 6028, 3926, 102, 0, 0, 0, 0, 0, 0, 0, 0]]),
 'token_type_ids': tensor([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
  0, 0, 0, 0, 0, 0, 0, 0]]),
 'attention_mask': tensor([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
  0, 0, 0, 0, 0, 0, 0, 0]])}
"""
```

[CLS] Sentence 1 [SEP] Sentence 2 [SEP]



三、实现

（二）模型实现

1、词向量处理：选择、词向量初始化

2、表示模型

3、编码（encoder）

4、交互模块

cross-sentence Attention

5、BERT实现

三、实现

（二）模型实现-词向量处理

词向量初始化

```
from gensim.models import KeyedVectors

#载入词向量
w2v_model = KeyedVectors.load_word2vec_format(data_args.w2v_file, binary=False)

def _init_word_embedding(self, state_dict=None, vocab_size=None, word_embedding_dim=None):
    if state_dict is None:
        self.word_embedding = nn.Embedding(vocab_size, word_embedding_dim, padding_idx=0)
    else:
        # 默认载入预训练好的词向量（且固定词向量），并将其第一个词作为填充词（以及其对应向量设为零向量）
        state_dict = torch.tensor(state_dict.vectors, dtype=torch.float32)
        state_dict[0] = torch.zeros(state_dict.size(-1))
        self.word_embedding = nn.Embedding.from_pretrained(state_dict, freeze=True, padding_idx=0)
```

设置Embedding是否固定

三、实现

（二）模型实现-词向量处理

词向量初始化

```
from gensim.models import KeyedVectors

#载入词向量
w2v_model = KeyedVectors.load_word2vec_format(data_args.w2v_file, binary=False)

def _init_word_embedding(self, state_dict=None, vocab_size=None, word_embedding_dim=None):
    if state_dict is None:
        self.word_embedding = nn.Embedding(vocab_size, word_embedding_dim, padding_idx=0)
    else:
        # 默认载入预训练好的词向量（且固定词向量），并将其第一个词作为填充词（以及其对应向量设为零向量）
        state_dict = torch.tensor(state_dict.vectors, dtype=torch.float32)
        state_dict[0] = torch.zeros(state_dict.size(-1))
        self.word_embedding = nn.Embedding.from_pretrained(state_dict, freeze=True, padding_idx=0)
```

设置Embedding是否固定

三、实现

（二）模型实现-表示模型

表示模型（前向传播）

```
def forward(
    self,
    text_a_input_ids,
    text_b_input_ids,
    text_a_attention_mask,
    text_b_attention_mask,
    labels=None
):
    text_a_vecs = self.word_embedding(text_a_input_ids)
    text_b_vecs = self.word_embedding(text_b_input_ids)

    text_a_vec = self.encoder(text_a_vecs, text_a_attention_mask)
    text_b_vec = self.encoder(text_b_vecs, text_b_attention_mask)

    pooler_output = torch.concat([text_a_vec, text_b_vec], dim=-1)
    logits = self.classifier(pooler_output)

    loss = None
    if labels is not None:
        loss_fct = nn.CrossEntropyLoss()
        loss = loss_fct(logits.view(-1, self.num_labels), labels.view(-1))

        return (loss, logits) if loss is not None else logits
```

Step 1： Look up 词向量

Step 2：两个 Query 分别通过 Encoder

Step 3：拼接两个 Query 的表示，再输入到分类器中

Step 4：训练阶段的 Loss 计算

三、实现

（二）模型实现-编码

Encoder (Neural Network)

```
class Encoder(nn.Module):

    def __init__(self, in_feat: int = 100, dropout_prob: float = 0.1):
        super().__init__()

MLP     self.dense1 = nn.Linear(in_feat, in_feat)
        self.dense2 = nn.Linear(in_feat, in_feat)
        self.dropout = nn.Dropout(dropout_prob)
        self.act = nn.Tanh()

    def forward(self, token_embeds, attention_mask):
        batch_size = token_embeds.size(0)

        # 简单地将文本的词向量求和，作为文本的表示
        x = torch.stack([token_embeds[i, attention_mask[i, :], :].sum(dim=0) for i in range(batch_size)], dim=0)

        x = self.act(self.dense1(x))
        x = self.act(self.dense2(self.dropout(x)))

        return x
```

三、实现

（二）模型实现-交互

Cross-Sentence Attention

以 Query A 关于 Query B 做 Attention 计算为例

```
in_feat = a.size(-1)

# a: [B, L1, H], b: [B, L2, H]
# 计算query A和query B之间的attention score，即 Key * Value
cross_attn = torch.matmul(a, b.transpose(1, 2))   # [B, L1, L2]

# 将填充位置的score设为-1e9，即不考虑填充位置的信息
row_attn = cross_attn.masked_fill((mask_b == False).unsqueeze(1), -1e9)
row_attn = row_attn.softmax(dim=2)    # [B, L1, L2]

# attention score * value
attn_a = torch.matmul(row_attn, b)      # [B, L1, H]

diff_a = a - attn_a
diff_b = b - attn_b

# 将原本的hidden state和attention得到的hidden state拼接，并经过线性变换降维
a = torch.cat([a, attn_a, diff_a, prod_a], dim=-1)    # [B, L1, 2*H]
a = self.act(self.dense(self.dropout(a)))                # [B, L1, H/2]
```

Step 1：QK^T 的计算

Step 2：遮掩 Query B 的填充位置的信息

Step 3：Softmax 计算

Step 4：$\overline{H}_1 = Softmax(QK^T) \cdot V$ 计算

Step 5：$\widetilde{H}_1 = Encoder_2([H_1; \overline{H}_1])$ 的计算

三、实现

（二）模型实现-Bert实现

BERT 模型

```
from transformes import BertForSequenceClassification

model = BertForSequenceClassification.from_pretrained('<MODEL_DIR>', num_labels=len(processor.get_labels()))
outputs = model(**inputs, return_dict=True)

"""
Returns transformers.modeling_outputs.SequenceClassifierOutput(
    loss: Classification Loss;
    logits: Classification score (Before SoftMax);
    hidden_states;
    attentions: Attention weights after the attention softmax
)
"""
```

四、其他思考

- 1、用更好的模型，如BERT-Large
- 2、更多的集成学习模型，如Baggin、Boosting
- 3、先验知识融入，如ConceptNet结构化知识库
- 4、新的预训练模型微调范式